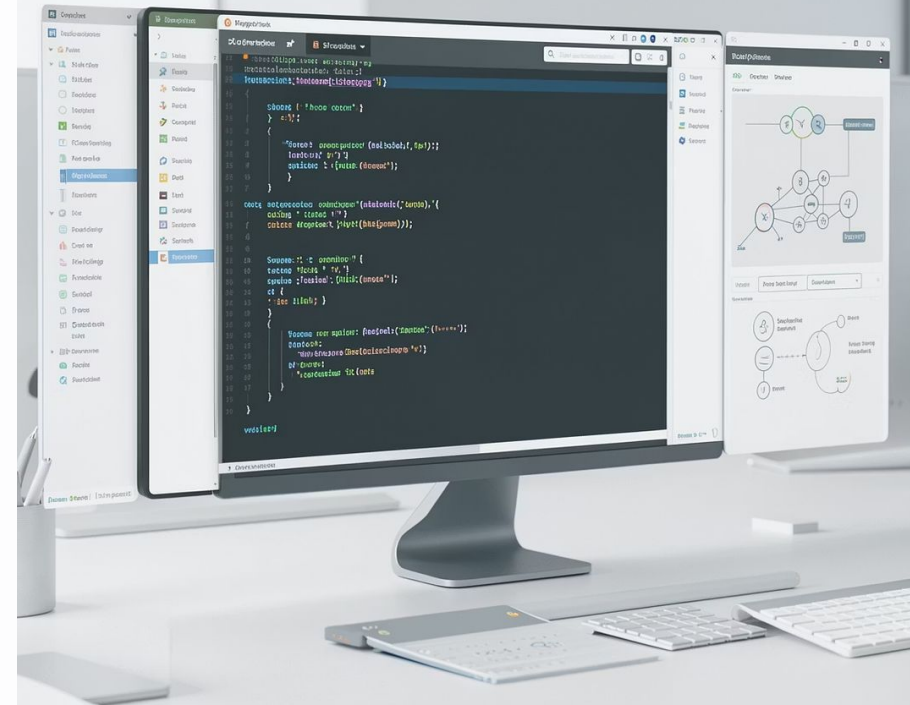


Testes Unitários: Pytest x Mocha

Comparação entre frameworks de testes em Python e JavaScript

 PYTHON

 JAVASCRIPT



O que são Testes Unitários?



Verificação Isolada

Testam funções, métodos ou componentes de forma isolada, sem dependências externas.



Detecção de Erros

Identificam erros rapidamente e evitam regressões no código ao longo do tempo.



Documentação Viva

Documentam o comportamento esperado do código, servindo como referência para toda a equipe.



Partes Pequenas

Verificam se pequenas partes de um programa funcionam corretamente de forma independente.

Por que Testar?

Detecção Antecipada

Encontre erros antes que cheguem à produção.

Confiabilidade

Maior qualidade e estabilidade do código entregue.

Refatoração Segura

Facilita manutenção sem medo de quebrar funcionalidades.

Automação

Automatiza a verificação contínua do sistema.



PYTHON

Pytest

Framework moderno, poderoso e com sintaxe extremamente legível para testar projetos Python.

- Sintaxe simples e legível
- Fixtures poderosas e parametrização
- Informações detalhadas em falhas

Exemplo Prático

```
# funcao.py
def somar(a, b):
    return a + b

# test_funcao.py
def test_somar():
    assert somar(2, 3) == 5

def test_somar_negativos():
    assert somar(-1, -1) == -2
```

Saída do Terminal

```
collected 2 items

test_funcao.py .. [100%]
2 passed in 0.03s
```

Pytest: Prós e Contras

✓ Fácil de Aprender e Usar

Curva de aprendizado baixa — ideal para iniciantes e projetos ágeis.

✓ Testes Legíveis e Diretos

A palavra-chave `assert` é nativa do Python, sem necessidade de bibliotecas extras.

✓ Grande Ecossistema de Plugins

Centenas de plugins disponíveis para cobertura, mock, relatórios e mais.

⚠ Plugins para Recursos Específicos

Alguns cenários avançados exigem instalação de plugins adicionais.



Mocha

Framework flexível e amplamente adotado no ecossistema JavaScript e Node.js.

- Estrutura baseada em `describe()` e `it()`
- Suporte forte a testes assíncronos
- Muito flexível e combinável com outras bibliotecas

Exemplo Prático

```
// funcao.js
function somar(a, b) {
  return a + b;
}

// test_funcao.js
const assert = require('assert');

describe('somar', function () {
  it('deve retornar 5', function () {
    assert.strictEqual(somar(2, 3), 5);
  });
  it('deve somar negativos', function () {
    assert.strictEqual(somar(-1, -1), -2);
  });
});
```

Saída do Terminal

```
somar
✓ deve retornar 5
✓ deve somar negativos

2 passing (8ms)
```

Mocha: Prós e Contras

✓ Muito Flexível e Organizado

A estrutura `describe/it` organiza testes em blocos hierárquicos claros.

✓ Forte Suporte a Assíncrono

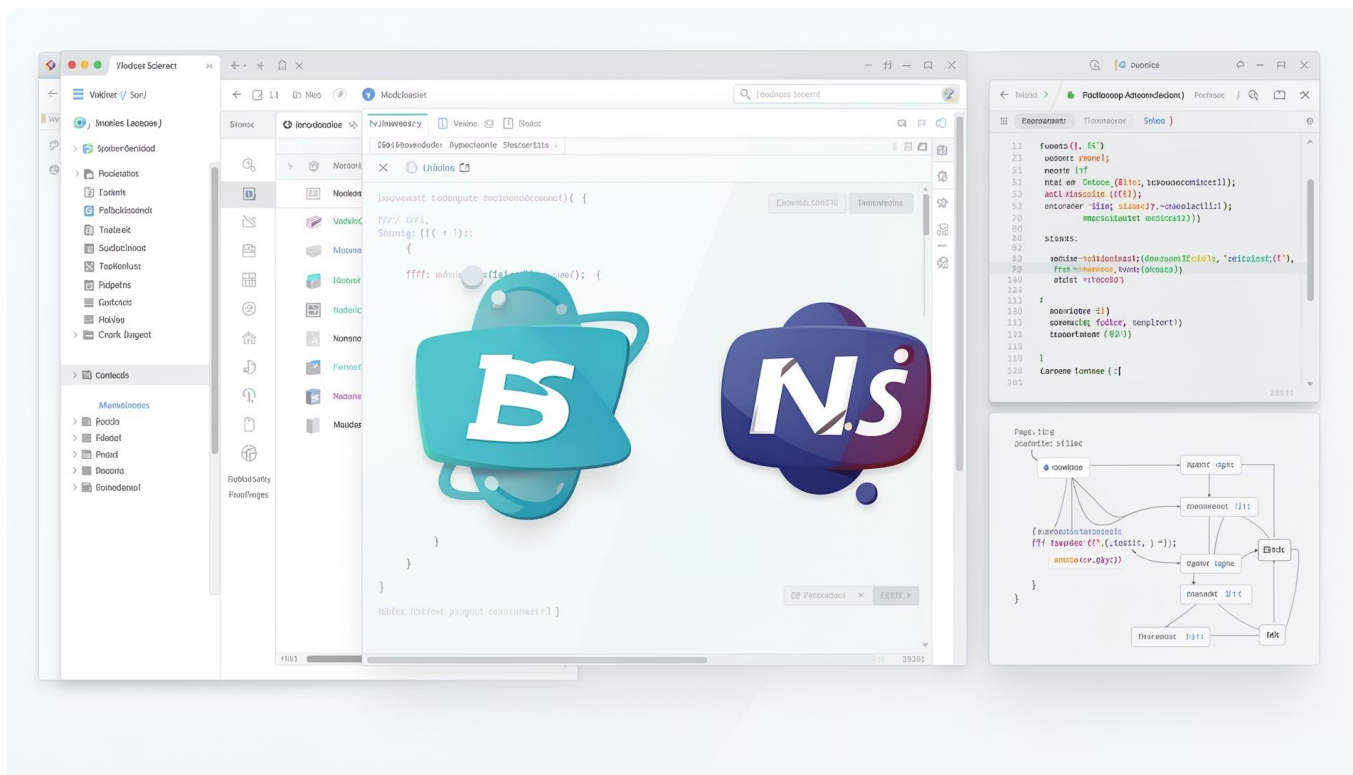
Suporte nativo a Promises, `async/await` e callbacks — ideal para Node.js.

✓ Amplamente Utilizado

Grande comunidade, abundância de exemplos e integração com CI/CD.

▣ Bibliotecas Adicionais

Asserções, mocks e espões precisam de libs como Chai, Sinon, etc.



Pytest vs Mocha: Comparação

Critério	Pytest (Python)	Mocha (JavaScript)
Linguagem	Python	JavaScript / Node.js
Sintaxe	Simples e direta (assert)	Estruturada (describe / it)
Asserções	Nativas do Python	Bibliotecas auxiliares (Chai)
Assíncrono	Suporte via plugins	Suporte nativo e robusto
Facilidade	Alta — curva suave	Alta / Média — requer configuração
Fixtures	Nativas e poderosas	Via beforeEach / afterEach

Quando Utilizar?

Escolha Pytest quando...

Projeto em Python

Backend, dados, IA, ciência de dados — Pytest é o padrão.

Sintaxe Simples é Prioridade

Ótimo para quem está aprendendo ou quer agilidade.

Fixtures e Parametrização

Cenários complexos com múltiplos casos de teste.

Escolha Mocha quando...

Projeto JavaScript / Node.js

APIs, servidores, front-end — Mocha é referência no ecossistema JS.

Flexibilidade é Essencial

Combine com Chai, Sinon e outras libs conforme a necessidade.

Testes Assíncronos

Suporte nativo a async/await e Promises é diferencial.

Conclusão

Testes = Qualidade

Testes unitários aumentam a qualidade e a confiabilidade de todo software.

Pytest — Simplicidade

Recursos poderosos com sintaxe elegante. Ideal para Python.

Mocha — Flexibilidade

Ecossistema rico e suporte assíncrono robusto. Ideal para JavaScript.

"Testar não é apenas encontrar erros: é garantir que o código continue funcionando como esperado."

